

DevOps

DevOps adalah kata kunci umum dalam pengembangan perangkat lunak. Tetapi apakah Anda baru mengenal industri ini atau telah berada di dalamnya selama bertahun-tahun, Anda harus tahu apa itu DevOps dan bagaimana hal itu dapat memengaruhi cara Anda bekerja. Dalam bacaan ini, Anda akan menjelajahi DevOps dan memeriksa budaya dan komponen umum.

Apa itu DevOps?

Definisi DevOps tergantung kepada siapa Anda bertanya. Tetapi bagi sebagian besar ahli, DevOps bermuara pada hal-hal berikut:

***DevOps** adalah pendekatan pengembangan perangkat lunak yang melibatkan kolaborasi tim pengembangan (Dev) dan operasi (Ops) untuk memberikan pembaruan perangkat lunak yang cepat dan berkualitas tinggi.*

Mari kita uraikan komponen kunci dari definisi ini.

Pengembangan

“Dev” di DevOps mengacu pada tim **pengembangan**. Tim ini menulis, mengedit, dan mendokumentasikan kode perangkat lunak dan memeliharanya dalam basis kode atau repositori.

Operasi TI

“Ops” di DevOps mengacu pada tim **operasi teknologi informasi (TI)**. Tim ini menyiapkan, mengelola, dan mengoptimalkan infrastruktur yang menjadi sandaran pengembangan, termasuk perangkat keras, perangkat lunak, jaringan, dan penyimpanan cloud. Tim juga menyebarkan pembaruan perangkat lunak kepada pengguna dan mendukung kode tersebut.

Kecepatan

Salah satu tujuan DevOps adalah untuk merilis kiriman, yang merupakan produk atau layanan yang merupakan bagian dari proyek, dengan cepat dan sering melalui penerapan yang cepat. Dengan DevOps, organisasi merilis pembaruan kecil namun berkelanjutan alih-alih perubahan besar yang jarang terjadi.

Kualitas

Hasil harus berkualitas tinggi, artinya memenuhi harapan pemangku kepentingan untuk kinerja, fungsionalitas, stabilitas, keandalan, ketersediaan, dan keamanan. Para pemangku kepentingan ini termasuk tim pengembangan dan operasi tetapi juga tim keamanan, tim lini bisnis, dan pengguna, di antara pihak-pihak lain.

Budaya

DevOps hanya berfungsi jika organisasi menganut budaya DevOps, dan landasan budaya tersebut adalah **kolaborasi**.

Biasanya, tim pengembangan dan operasi bekerja secara terpisah di silo masing-masing. Namun dengan DevOps, mereka bekerja sama dan menjaga komunikasi terbuka sepanjang **siklus hidup pengembangan perangkat lunak (SDLC)**. Mereka berbagi informasi secara bebas dan transparan serta bekerja sama untuk memastikan bahwa pengembang menerima **masukan cepat** tentang masalah perangkat lunak apa pun.

Tim juga **berbagi tanggung jawab** atas kualitas produk.

- Pengembang tidak hanya membangun kode tetapi mengujinya dan membantu mempertahankannya dari waktu ke waktu; mereka tidak menyerahkan tugas-tugas ini kepada tim operasi.
- Dan personel operasi berpartisipasi dalam diskusi perencanaan untuk pembaruan perangkat lunak. Dengan demikian, mereka dapat memastikan bahwa pengembang membangun pemantauan, pencatatan, pengujian keamanan, dan fitur lain yang dibutuhkan sistem.

Dengan berkolaborasi di seluruh SDLC, tim lebih memahami kebutuhan satu sama lain. Pada gilirannya, mereka dapat menyesuaikan praktik kerja mereka untuk memenuhi kebutuhan tersebut, meningkatkan hubungan tim dan menghilangkan hambatan yang menunda rilis.

Komponen

Mari kita bahas komponen DevOps lainnya yang umum diakui.

Ketangkasan

DevOps biasanya melibatkan pendekatan tangkas (agile approach) untuk pengembangan perangkat lunak. Untuk menjelaskan pendekatan tangkas, mari kita bahas dulu pendahulunya, pendekatan air terjun (waterfall approach).

Pendekatan **air terjun** bersifat linier. Tim mengikuti rencana yang panjang dan terperinci dengan fase berurutan yang berbeda seperti perencanaan, desain, pengembangan, pengujian, dan dukungan. Rencana ini menetapkan tenggat waktu untuk setiap fase dan tanggung jawab setiap anggota tim, dan tujuannya adalah untuk melaksanakan rencana tersebut setepat mungkin.

Tetapi pendekatan ini menghadirkan beberapa masalah untuk pengembangan perangkat lunak.

- Saat proyek bergerak maju, pengembang mungkin menghadapi hambatan atau memikirkan ide-ide baru yang lebih baik, dan klien mungkin mengubah persyaratan mereka. Namun rencana yang kaku membuat adaptasi menjadi sulit dan terkadang mahal.
- Lebih buruk lagi, klien dan pengguna biasanya tidak melihat produk sampai tahap pengujian, sehingga pengembang tidak menerima umpan balik sampai saat itu. Mengatasi bug dan masalah klien yang di akhir proses bisa lebih mahal dan memakan waktu daripada sebelumnya dalam pengembangan.

Sebaliknya, **pengembangan tangkas** bersifat berulang. Alih-alih menyelesaikan proyek dalam fase yang lebih panjang dan berbeda, tim bekerja dalam iterasi singkat, yang dikenal sebagai sprint, yang biasanya masing-masing berdurasi beberapa minggu. Tujuan dari setiap sprint adalah untuk menghasilkan kiriman kecil dimana klien dapat memberikan umpan balik dengan cepat. Dengan cara ini, pengembang dapat mempelajari kekhawatiran klien sejak dini dan memperhitungkannya di masa mendatang. Selain itu, pengujian dilakukan di setiap sprint, bukan di akhir waktu proyek, sehingga tim dapat mengidentifikasi dan memperbaiki bug lebih cepat.

Seperti proyek air terjun, proyek tangkas masih memiliki tujuan keseluruhan. Namun, rencananya tidak terlalu terperinci dan lebih fleksibel untuk mengakomodasi ide dan persyaratan baru.

Singkatnya, pendekatan tangkas sangat ideal untuk DevOps.

- Berfokus pada sprint singkat, kiriman kecil, dan pengujian berkelanjutan sangat cocok dengan tujuan DevOps untuk pembaruan perangkat lunak berkualitas tinggi yang sering.
- Pengembang dan personel operasi dapat dengan cepat beradaptasi dengan perubahan tuntutan proyek dan standar kualitas.

Layanan Mikro

Untuk mempercepat penerapan, DevOps memerlukan desain aplikasi yang berfokus pada layanan mikro. Layanan **mikro** adalah komponen aplikasi kecil yang berfungsi secara independen dan dapat bekerja dengan komponen lain untuk membentuk aplikasi yang lengkap.

Dalam pengembangan tradisional, pengembang membangun semua bagian aplikasi menjadi satu bagian monolitik besar. Saat menambahkan fitur atau memperbaiki bug, pengembang harus menyebarkan ulang seluruh aplikasi.

Tetapi ketika menggunakan layanan mikro, pengembang hanya perlu mengedit komponen yang relevan, bukan seluruh aplikasi. Perubahan skala kecil ini membuat pembaruan lebih cepat dan mengurangi risiko kerusakan aplikasi.

Otomatisasi

DevOps melibatkan penggunaan alat untuk **mengotomatiskan** pergerakan kode melalui SDLC.

Misalnya dalam pengujian. Kode harus menjalani tingkat pengujian yang berbeda sepanjang siklus hidupnya, dan berbagai alat dapat secara otomatis menjalankan pengujian yang sesuai untuk setiap tahap. Alat juga dapat menyiapkan dan mengonfigurasi komponen infrastruktur, memantau kinerja dan masalah keamanan, mengidentifikasi komponen atau aset yang tidak kompatibel, dan menerapkan kode ke produksi.

Otomatisasi mengurangi waktu penerapan dan kemungkinan kesalahan manusia. Ini juga membebaskan pengembang dan personel operasi untuk fokus pada pengkodean, pemecahan masalah, dan tugas-tugas penting lainnya untuk meningkatkan produk.

Integrasi berkelanjutan dan pengiriman berkelanjutan

Integrasi berkelanjutan dan pengiriman berkelanjutan (CI/CD) adalah dua proses otomatis yang penting untuk DevOps.

- **Integrasi berkelanjutan (CI)** adalah proses di mana pengembang secara teratur mengintegrasikan potongan-potongan kecil kode yang baru ditulis ke dalam basis kode utama mereka. Kode menjalani pengujian otomatis, dan manajer basis kode meninjau dan menyetujuinya sebelum integrasi.
- **Pengiriman berkelanjutan (Continuous Delivery (CD))** adalah yang berikutnya. Ini adalah proses yang mengirimkan kode baru ini ke lingkungan praproduksi untuk memverifikasi bahwa kode tersebut siap untuk digunakan secara cepat dan aman ke produksi. Di sini, kode menjalani pengujian dan peninjauan tambahan, dan kemudian dipindahkan ke produksi.

CI/CD sangat berharga untuk DevOps.

- CI/CD mengotomatiskan pengujian, menghemat waktu yang dihabiskan untuk menguji kode secara manual dan memberikan umpan balik instan kepada pengembang tentang bug.
- CI/CD mengotomatiskan pemeriksaan keamanan siber seperti pemindaian aplikasi dan kerentanan, yang merupakan bagian standar **DevSecOps** (DevOps dengan keamanan bawaan).
- Pengujian, pemindaian, dan peninjauan yang sering juga berarti kode berkualitas lebih tinggi.
- CI/CD mengotomatiskan penerapan ke lingkungan praproduksi.

Dengan CI/CD, pembaruan dapat dilakukan dengan lebih cepat dan dengan upaya yang lebih sedikit.